



InterSystems Implementation Reference for Third-party Software

Version 2026.1
2026-04-20

InterSystems Implementation Reference for Third-party Software

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Overview of Java Third-party Software	1
2 JDBC Driver Support	3
2.1 JDBC and the InterSystems JDBC Driver	3
2.1.1 Installation and Configuration	3
2.2 JDBC Driver Compliance	4
2.2.1 Required java.sql Interfaces	4
2.2.2 Optional java.sql Interfaces	5
2.2.3 java.sql Exceptions	5
2.2.4 Required javax.sql Interfaces	6
2.2.5 Optional javax.sql Interfaces	6
2.3 Variants and Unsupported Optional Methods	6
2.3.1 CallableStatement: Unsupported Methods	7
2.3.2 Connection: Unsupported or Restricted Methods	7
2.3.3 DatabaseMetaData: Variant Methods	9
2.3.4 Driver: Unsupported Methods	9
2.3.5 PreparedStatement: Unsupported Methods	10
2.3.6 ResultSet: Unsupported or Restricted Methods	10
2.3.7 Statement: Unsupported or Restricted Methods	11
2.4 InterSystems Enhancements and Extensions	12
2.4.1 CallableStatement getBinaryStream() Extension Method	12
2.4.2 ConnectionPoolDataSource Extensions and Enhancements	13
2.4.3 DataSource Extensions and Enhancements	14
3 Hibernate Support	19
3.1 Requirements and Configuration	19
4 R Gateway Support	21
4.1 Installing R and RServe	21
4.2 Running the R Gateway	22

List of Figures

Figure 4–1: R Gateway System	21
------------------------------------	----

1

Overview of Java Third-party Software

See the [Table of Contents](#) for a detailed listing of the subjects covered in this document.

InterSystems provides native implementations of the following third-party drivers and interfaces:

The InterSystems JDBC Driver

The InterSystems JDBC Driver is a fully compliant type 4 implementation of the JDBC standard.

See [JDBC Driver Support](#) for detailed information on JDBC driver compliance and enhancements, including the level of support for all optional features and a list of all InterSystems IRIS-specific additional features.

Also see the following related documentation:

- [Using Java with InterSystems Software](#) provides a full description of features and usage.

The InterSystems Hibernate Dialect

The InterSystems Hibernate Dialect is an implementation of the Hibernate dialect interface. Since every vendor's implementation of SQL is slightly different, the Hibernate dialect interface allows vendors to create custom Hibernate mappings for a specific database.

See [Hibernate Support](#) for details.

The InterSystems R Gateway

The InterSystems R gateway implements an embeds the third-party REngine library as a Java client to Rserve, which is a TCP/IP server allowing other programs to use facilities of R.

See [R Gateway Support](#) for details.

2

JDBC Driver Support

The InterSystems IRIS® JDBC Driver is a fully compliant type 4 implementation of the JDBC 4.2 standard. This section lists all classes and interfaces of the JDBC 4.2 API, indicates the level of support for each one, and describes all InterSystems-specific features. The following topics are discussed:

- [JDBC and the InterSystems JDBC Driver](#) — provides an overview and resource links for the JDBC driver.
- [JDBC Driver Compliance](#) — lists all classes and interfaces specified by the JDBC standard, and indicates the current level of support.
- [Variants and Unsupported Optional Methods](#) — provides details on classes that include permitted variances from the standard.
- [InterSystems Enhancements and Extensions](#) — lists and discusses InterSystems extensions to the standard JDBC API.

[Connecting Your Application to InterSystems IRIS](#) provides instructions, including sample code, for connecting to an InterSystems IRIS server from a Java application using JDBC.

2.1 JDBC and the InterSystems JDBC Driver

The Java JDBC API is the industry standard for vendor-neutral database connectivity. It provides a reliable way for Java applications to connect to data sources on any supported platform and to query or perform operations on them with SQL.

InterSystems JDBC is implemented in a type 4 driver to deliver the highest possible performance. *Type 4* means that it is a direct-to-database pure Java driver, installed inside the client JVM and requiring no external software support. It is fully compliant with the JDBC 4.2 API specification, supporting all required interfaces and adhering to all JDBC 4.2 guidelines and requirements. InterSystems IRIS supports all features except SQL Exception handling enhancements, National Character Set conversions, and the XML data type.

See [Using Java with InterSystems Software](#) for a full description of API features and usage. That book also provides an overview of all InterSystems IRIS Java technologies enabled by the JDBC driver (see “[InterSystems Java Connectivity Options](#)”).

2.1.1 Installation and Configuration

The InterSystems JDBC driver is included in the standard InterSystems IRIS installation package. No extra installation or setup procedures are required. See “[Client-Server Configuration](#)” in [Using Java with InterSystems Software](#) for information on client requirements and usage.

For more information about how to make a connection between InterSystems IRIS and an application using the JDBC driver, see [“Connecting Your Application to InterSystems IRIS.”](#)

If you want to use the JDBC driver on a system that does not have InterSystems IRIS installed, you can download the driver package from the [InterSystems IRIS Drivers page](#) on GitHub.

2.2 JDBC Driver Compliance

This section provides information on the level of support for each JDBC interface.

2.2.1 Required java.sql Interfaces

The following interfaces must be implemented. Some classes contain methods that are optional if the implementation depends on a feature that the database does not support. The *standard implementation* annotation indicates that the generic implementation of the class has been used without alteration:

- `java.sql.CallableStatement` — implemented with some permitted variances (see [“CallableStatement: Unsupported Methods”](#) and [“CallableStatement getBinaryStream\(\) Extension Method”](#)).
- `java.sql.ClientInfoStatus` — *standard implementation*.
- `java.sql.Connection` — implemented with some permitted variances (see [Connection: Unsupported or Restricted Methods](#)).
- `java.sql.DatabaseMetaData` — implemented with some permitted variances (see [“DatabaseMetaData: Variant Methods”](#)).
- `java.sql.Date` — *standard implementation*.
- `java.sql.Driver` — implemented with some permitted variances (see [“Driver: Unsupported Methods”](#)).
- `java.sql.DriverManager` — *standard implementation*.
- `java.sql.DriverPropertyInfo` — *standard implementation*.
- `java.sql.ParameterMetaData` — all methods fully supported.
- `java.sql.PreparedStatement` — implemented with some permitted variances (see [“PreparedStatement: Unsupported Methods”](#)).
- `java.sql.ResultSet` — implemented with some permitted variances (see [“ResultSet: Unsupported or Restricted Methods”](#)).
- `java.sql.ResultSetMetaData` — all methods fully supported.
- `java.sql.RowIdLifetime` — *standard implementation*.
- `java.sql.SQLPermission` — *standard implementation*.
- `java.sql.Statement` — implemented with some permitted variances (see [“Statement: Unsupported or Restricted Methods”](#)).
- `java.sql.Time` — *standard implementation*.
- `java.sql.Timestamp` — *standard implementation*.
- `java.sql.Types` — *standard implementation*.
- `java.sql Wrapper` — all methods fully supported.

2.2.2 Optional java.sql Interfaces

All optional java.sql interfaces are listed below. *Italicized* items are not implemented:

- *java.sql.Array*
- java.sql.Blob — all methods fully supported.
- java.sql.Clob — all methods fully supported.
- java.sql.NClob — all methods fully supported.
- *java.sql.Ref*
- java.sql.RowId — all methods fully supported.
- java.sql.Savepoint — all methods fully supported.
- *java.sql.SQLData*
- *java.sql.SQLInput*
- *java.sql.SQLOutput*
- *java.sql.XML*
- *java.sql.Struct*

2.2.3 java.sql Exceptions

The InterSystems JDBC driver throws only the following exceptions:

- java.sql.BatchUpdateException
- java.sql.SQLException
- java.sql.SQLWarning

The following exceptions are listed here for completeness, but are not required and are never used:

- *DataTruncation*
- *SQLClientInfoException*
- *SQLDataException*
- *SQLFeatureNotSupportedException*
- *SQLIntegrityConstraintViolationException*
- *SQLInvalidAuthorizationSpecException*
- *SQLNonTransientConnectionException*
- *SQLNonTransientException*
- *SQLRecoverableException*
- *SQLSyntaxErrorException*
- *SQLTimeoutException*
- *SQLTransactionRollbackException*
- *SQLTransientConnectionException*

- *SQLException*

2.2.4 Required javax.sql Interfaces

The following required interfaces are supported. The *standard implementation* annotation indicates that the generic implementation of the class has been used without alteration:

- `javax.sql.ConnectionEvent` — *standard implementation*.
- `javax.sql.DataSource` — implemented with enhancements and additional methods (see “[DataSource Extensions and Enhancements](#)” for details).
- `javax.sql.RowSetEvent` — *standard implementation*.
- `javax.sql.StatementEvent` — *standard implementation*.

2.2.5 Optional javax.sql Interfaces

All optional `javax.sql` interfaces are listed below. *Italicized* items are not implemented:

- *javax.sql.CommonDataSource* — not implemented. Use `javax.sql.DataSource` instead (see “[DataSource Extensions and Enhancements](#)” for related information).
- `javax.sql.ConnectionEventListener` — all methods fully supported.
- `javax.sql.ConnectionPoolDataSource` — implemented with variants and additional methods (see “[ConnectionPoolDataSource Extensions and Enhancements](#)” for details).
- `javax.sql.PooledConnection` — all methods fully supported.
- *javax.sql.Rowset*
- *javax.sql.RowSetInternal*
- *javax.sql.RowSetListener*
- *javax.sql.RowSetMetaData*
- *javax.sql.RowSetReader*
- *javax.sql.RowSetWriter*
- *javax.sql.StatementEventListener*
- *javax.sql.XAConnection*
- *javax.sql.XADataSource*

2.3 Variants and Unsupported Optional Methods

The following interfaces have optional methods that the InterSystems JDBC driver does not support, or methods implemented in a non-standard manner:

- [CallableStatement: Unsupported Methods](#)
- [Connection: Unsupported or Restricted Methods](#)
- [DatabaseMetaData: Variant Methods](#)

- [Driver: Unsupported Methods](#)
- [PreparedStatement: Unsupported Methods](#)
- [ResultSet: Unsupported or Restricted Methods](#)
- [Statement: Unsupported or Restricted Methods](#)

2.3.1 CallableStatement: Unsupported Methods

Unsupported Optional Methods

`java.sql.CallableStatement` does not support the following optional methods:

- **getArray()**


```
Array getArray(int i)
Array getArray(String parameterName)
```
- **getObject()**


```
Object getObject(int i, java.util.Map map)
Object getObject(String parameterName, java.util.Map map)
```
- **getRef()**


```
Ref getRef(int i)
Ref getRef(String parameterName)
```
- **getRowId() and setRowId()**


```
java.sql.RowId getRowId(int i)
java.sql.RowId getRowId(String parameterName)

void setRowId(String parameterName, java.sql.RowId x)
```
- **getURL() and setURL()**


```
java.net.URL getURL(int i)
java.net.URL getURL(String parameterName)

void setURL(String parameterName, java.net.URL val)
```
- **getSQLXML() and setSQLXML()**


```
java.sql.SQLXML getSQLXML(int parameterIndex)
java.sql.SQLXML getSQLXML(String parameterName)

void setSQLXML(String parameterName, java.sql.SQLXML xmlObject)
```

Note: The `java.sql.CallableStatement` class also has one InterSystems extension method, which is discussed elsewhere (see “[CallableStatement getBinaryStream\(\) Extension Method](#)”).

2.3.2 Connection: Unsupported or Restricted Methods

Unsupported Optional Methods

The InterSystems implementation of `java.sql.Connection` does not support the following optional methods:

- **abort()**


```
void abort(Executor executor)
```

- **createArrayOf()**

```
java.sql.Array createArrayOf(String typeName, Object[] elements)
```

- **createBlob()**

```
Blob createBlob()
```

- **createClob()**

```
Clob createClob()
```

- **createNClob()**

```
java.sql.NClob createNClob()
```

- **createSQLXML()**

```
java.sql.SQLXML createSQLXML()
```

- **createStruct()**

```
java.sql.Struct createStruct(String typeName, Object[] attributes)
```

- **getTypeMap()**

```
java.util.Map getTypeMap()
```

- **setTypeMap()**

```
void setTypeMap(java.util.Map map)
```

Optional Methods with Restrictions

The following optional `java.sql.Connection` methods are implemented with restrictions or limitations:

- **prepareCall()**

Only `TYPE_FORWARD_ONLY` is supported for *resultSetType*. Only `CONCUR_READ_ONLY` is supported for *resultSetConcurrency*.

```
java.sql.CallableStatement prepareCall(String sql, int resultSetType, int resultSetConcurrency)
```

- **setReadOnly()**

A no-op (the InterSystems IRIS driver does not support `READ_ONLY` mode)

```
void setReadOnly(Boolean readOnly)
```

- **setCatalog()**

A no-op (the InterSystems IRIS driver does not support catalogs)

```
void setCatalog(String catalog)
```

- **setTransactionIsolation()**

Only `TRANSACTION_READ_COMMITTED` and `TRANSACTION_READ_UNCOMMITTED` are supported for *level*.

```
void setTransactionIsolation(int level)
```

The following `java.sql.Connection` methods do not support `CLOSE_CURSORS_AT_COMMIT` for *resultSetHoldability*:

- **createStatement()**

```
java.sql.Statement createStatement(int resultSetType, int result, int resultSetHoldability)
```

- **prepareCall()**

```
java.sql.CallableStatement prepareCall(String sql,
                                         int resultSetType,
                                         int resultSetConcurrency,
                                         int resultSetHoldability)
```

- **prepareStatement()**

```
java.sql.PreparedStatement prepareStatement(String sql,
                                             int resultSetType,
                                             int resultSetConcurrency,
                                             int resultSetHoldability)
```

InterSystems IRIS currently supports only zero or one Auto Generated Keys. An exception is thrown if the `java.sql.Connection` methods below provide *columnIndexes* or *columnNames* arrays whose lengths are not equal to one.

- **prepareStatement()**

```
java.sql.PreparedStatement prepareStatement(String sql, int[] columnIndexes)
java.sql.PreparedStatement prepareStatement(String sql, String[] columnNames)
```

2.3.3 DatabaseMetaData: Variant Methods

Variant Methods

`java.sql.DatabaseMetaData` is fully supported, but has methods that vary from the JDBC standard due to InterSystems-specific handling of their return values. The following methods are affected:

- **supportsMixedCaseQuotedIdentifiers()**

InterSystems IRIS returns `false`, which is not JDBC compliant.

```
boolean supportsMixedCaseQuotedIdentifiers()
```

- **getIdentifierQuoteString()**

If delimited id support is turned on, InterSystems IRIS returns `"` (double quote character), which is what a JDBC compliant driver should return; otherwise InterSystems IRIS returns a space.

```
String getIdentifierQuoteString()
```

2.3.4 Driver: Unsupported Methods

Unsupported Optional Method

`java.sql.Driver` does not support the following optional method:

- **getParentLogger()**

```
void getParentLogger()
```

2.3.5 PreparedStatement: Unsupported Methods

Unsupported Optional Methods

java.sql.PreparedStatement does not support the following optional methods:

- **setArray()**

void setArray(int i, Array x)
- **setRef()**

void setRef(int i, Ref x)
- **setRowId()**

void setRowId(int parameterIndex, RowId x)
- **setSQLXML()**

void setSQLXML(int parameterIndex, SQLXML xmlObject)
- **setUnicodeStream()**
Deprecated in Java JDK specification.

void setUnicodeStream(int i, InputStream x, int length)
- **setURL()**

void setURL(int i, java.net.URL x)

2.3.6 ResultSet: Unsupported or Restricted Methods

Optional Method with Restrictions

InterSystems IRIS does not support TYPE_SCROLL_SENSITIVE result set types. The following method is implemented with restrictions:

- **setFetchDirection()**

Does not support ResultSet.FETCH_REVERSE (instead, use **afterLast** to move the result set's cursor to after the last row, and use **previous** to scroll backwards).

void setFetchDirection(int direction)

Unsupported Optional Methods

java.sql.ResultSet does not support the following optional methods:

- **getArray()**

Array getArray(int i)
Array getArray(String colName)
- **getCursorName()**

String getCursorName()

- **getObject()**

```
Object getObject(int i, java.util.Map map)
Object getObject(String colName, java.util.Map map)
```

- **getRef()**

```
Ref getRef(int i)
Ref getRef(String colName)
```

- **getHoldability()**

```
int getHoldability()
```

- **getUnicodeStream()**

Deprecated in Java JDK specification.

```
java.io.InputStream getUnicodeStream(int i)
java.io.InputStream getUnicodeStream(String colName)
```

- **getURL()**

```
java.net.URL getURL(int i)
java.net.URL getURL(String colName)
```

- **updateArray()**

```
void updateArray(int i, Array x)
void updateArray(String colName, Array x)
```

- **updateRef()**

```
void updateRef(int i, Ref x)
void updateRef(String colName, Ref x)
```

2.3.7 Statement: Unsupported or Restricted Methods

Unsupported Optional Methods

java.sql.Statement does not support the following optional methods:

- **cancel()**

```
void cancel()
```

- **closeOnCompletion()**

```
void closeOnCompletion()
```

- **isCloseOnCompletion()**

```
boolean isCloseOnCompletion()
```

Optional Methods with Restrictions

The following optional java.sql.Statement methods are implemented with restrictions or limitations:

- **getResultSetHoldability()**

Only `HOLD_CURSORS_OVER_COMMIT`

```
int getResultSetHoldability()
```

- **setCursorName()**

A no-op.

```
void setCursorName(String name)
```

- **setEscapeProcessing()**

A no-op (does not apply)

```
void setEscapeProcessing(Boolean enable)
```

- **setFetchDirection()**

Does not support `ResultSet.FETCH_REVERSE` (instead, use **afterLast** to move the result set's cursor to after the last row, and use **previous** to scroll backwards).

```
void setFetchDirection(int direction)
```

InterSystems IRIS currently supports only zero or one auto-generated key. An exception is thrown if the `java.sql.Statement` methods below provide *columnIndexes* or *columnNames* arrays whose lengths are not equal to one:

- **execute()**

```
boolean execute(String sql, int[] columnIndexes)
boolean execute(String sql, String[] columnNames)
```

- **executeUpdate()**

```
int executeUpdate(String sql, int[] columnIndexes)
int executeUpdate(String sql, String[] columnNames)
```

2.4 InterSystems Enhancements and Extensions

The following classes provide additional InterSystems-specific extension methods:

- [CallableStatement getBinaryStream\(\) Extension Method](#)
- [ConnectionPoolDataSource Extensions and Enhancements](#) discusses `com.intersystems.jdbc.IRISConnectionPoolDataSource`, which is the InterSystems implementation of the `javax.sql.ConnectionPoolDataSource` interface.
- [DataSource Extensions and Enhancements](#) discusses `com.intersystems.jdbc.IRISDataSource`, which is the InterSystems implementation of `javax.sql.DataSource`.

2.4.1 CallableStatement getBinaryStream() Extension Method

`java.sql.CallableStatement` implements the following additional InterSystems-specific extension method:

- **getBinaryStream()**

Retrieves the value of the designated parameter (where *i* is the index of the parameter) as a `java.io.InputStream` object.

```
java.io.InputStream getBinaryStream(int i)
```

This method is a complement to the standard `setBinaryStream()` method, and an alternative to `getCharacterStream()` (which returns `java.io.Reader`).

2.4.2 ConnectionPoolDataSource Extensions and Enhancements

The `com.intersystems.jdbc.IRISConnectionPoolDataSource` class fully implements the `javax.sql.ConnectionPoolDataSource` interface. This class does not inherit the methods of `javax.sql.CommonDataSource`, which is not supported by the InterSystems JDBC driver.

Restricted Method

`getPooledConnection()` is implemented because it is required by the JDBC standard, but the InterSystems IRIS implementation should never be called directly. InterSystems IRIS driver connections must always be obtained by calling the `getConnection()` method. (See “[Using a Connection Pool](#)” in *Using Java with InterSystems Software* for more information).

- **getPooledConnection()**

```
javax.sql.PooledConnection getPooledConnection()
javax.sql.PooledConnection getPooledConnection(String usr,String pwd)
```

CAUTION: Calling applications should never use the `getPooledConnection()` methods or the `PooledConnection` class. InterSystems IRIS driver connections must always be obtained by calling the `getConnection()` method (which is inherited from `IRISDataSource`). The InterSystems IRIS driver provides pooling transparently through the `java.sql.Connection` object that it returns.

`IRISConnectionPoolDataSource` inherits from `IRISDataSource` (see “[DataSource Extensions and Enhancements](#)”), which provides additional InterSystems extension methods.

2.4.2.1 ConnectionPoolDataSource Extension Methods

`IRISConnectionPoolDataSource` also supports the following additional InterSystems IRIS-only management methods (see “[Using a Connection Pool](#)” in *Using Java with InterSystems Software* for more information):

- **restartConnectionPool()**

Restarts a connection pool. Closes all physical connections, and empties the connection pool.

```
void restartConnectionPool()
```

- **getPoolCount()**

Returns the current number of entries in the connection pool.

```
int getPoolCount()
```

- **setMaxPoolSize()**

Sets a maximum connection pool size. If the maximum size is not set, it defaults to 40.

```
void setMaxPoolSize(int max)
```

- **getMaxPoolSize()**

Returns the current maximum connection pool size

```
int getMaxPoolSize()
```

2.4.3 DataSource Extensions and Enhancements

The `com.intersystems.jdbc.IRISDataSource` class fully implements the `javax.sql.DataSource` interface. This class does not inherit the methods of `javax.sql.CommonDataSource`, which is not supported by the InterSystems JDBC driver.

Enhanced Required Method

The InterSystems IRIS implementation of this method is enhanced to provide automatic, transparent connection pooling. (See “[Using a Connection Pool](#)” in *Using Java with InterSystems Software* for more information).

- **getConnection()**

```
java.sql.Connection getConnection()  
java.sql.Connection getConnection(String usr,String pwd)
```

2.4.3.1 DataSource Extension Methods

In addition to the methods defined by the interface, `IRISDataSource` also implements the following methods that can be used to get or set `DataSource` properties supported by InterSystems IRIS. (See “[Connection Parameter Options](#)” in *Using Java with InterSystems Software* for more information).

- **getConnectionSecurityLevel()**

Returns an *int* representing the current Connection Security Level setting.

```
int getConnectionSecurityLevel()
```

- **getDatabaseName()**

Returns a *String* representing the current database (InterSystems IRIS namespace) name.

```
String getDatabaseName()
```

- **getDataSourceName()**

Returns a *String* representing the current data source name.

```
String getDataSourceName()
```

- **getDefaultTransactionIsolation()**

Gets the current default transaction isolation.

```
int getDefaultTransactionIsolation()
```

- **getDescription()**

Returns a *String* representing the current description.

```
String getDescription()
```

- **getEventClass()**

Returns a *String* representing an Event Class object.

```
String getEventClass()
```

- **getKeyRecoveryPassword()**
Returns a String representing the current Key Recovery Password setting.
`getKeyRecoveryPassword()`
- **getNodeDelay()**
Returns a boolean representing a current TCP_NODELAY option setting.
`boolean getNodeDelay()`
- **getPassword()**
Returns a String representing the current password.
`String getPassword()`
- **getPortNumber()**
Returns an *int* representing the current port number.
`int getPortNumber()`
- **getServerName()**
Returns a String representing the current server name.
`String getServerName()`
- **getServicePrincipalName()**
Returns a String representing the current Service Principal Name setting.
`String getServicePrincipalName()`
- **getSSLConfigurationName()**
Returns a String representing the current TLS Configuration Name setting.
`getSSLConfigurationName()`
- **getURL()**
Returns a String representing a current URL for this connection.
`String getURL()`
- **getUser()**
Returns a String representing the current username.
`String getUser()`
- **setConnectionSecurityLevel()**
Sets the connection security level

Sets the Connection Security Level for this DataSource object.
- **setDatabaseName()**

Sets the database name (InterSystems IRIS namespace) for this connection.

```
void setDatabaseName(String dn)
```

- **setDataSourceName()**

Sets the data source name for this connection. `DataSourceName` is an optional setting and is not used by `IRISDataSource` to connect.

```
void setDataSourceName(String dsn)
```

- **setDefaultTransactionIsolation()**

Sets the default transaction isolation level.

```
void setDefaultTransactionIsolation(int level)
```

- **setDescription()**

Sets the description for this connection. `Description` is an optional setting and is not used by `IRISDataSource` to connect.

```
void setDescription(String d)
```

- **setEventClass()**

Sets the Event Class for this connection. The Event Class is a mechanism specific to InterSystems IRIS JDBC. It is completely optional, and the vast majority of applications will not need this feature.

The InterSystems JDBC server will dispatch to methods implemented in a class when a transaction is about to be committed and when a transaction is about to be rolled back. The class in which these methods are implemented is referred to as the “event class.” If an event class is specified during login, then the JDBC server will dispatch to **%OnTranCommit** just prior to committing the current transaction and will dispatch to **%OnTranRollback** just prior to rolling back (aborting) the current transaction. User event classes should extend **%ServerEvent**. The methods do not return any values and cannot abort the current transaction.

```
void setEventClass(String e)
```

- **setKeyRecoveryPassword()**

Sets the Key Recovery Password for this connection.

```
setKeyRecoveryPassword(java.lang.String password)
```

- **setLogFile()**

Unconditionally sets the log file name for this connection.

```
setLogFile(java.lang.String logFile)
```

- **setNodeDelay()**

Sets the `TCP_NODELAY` option for this connection. Toggling this flag can affect the performance of the application. If not set, it defaults to `true`.

```
void setNodeDelay(boolean nd)
```

- **setPassword()**

Sets the password for this connection.

```
void setPassword(String p)
```

- **setPortNumber()**

Sets the port number for this connection

```
void setPortNumber(int pn)
```

- **setServerName()**

Sets the server name for this connection.

```
void setServerName(String sn)
```

- **setServicePrincipalName()**

Sets the Service Principal Name for this connection.

```
void setServicePrincipalName(String name)
```

- **setSSLConfigurationName()**

Sets the TLS Configuration Name for this connection.

```
setSSLConfigurationName(java.lang.String name)
```

- **setURL()**

Sets the URL for this connection.

```
void setURL(String u)
```

- **setUser()**

Sets the username for this connection.

```
void setUser(String u)
```


3

Hibernate Support

The InterSystems Hibernate Dialect is InterSystems implementation of the Hibernate dialect interface. Since every vendor's implementation of SQL is slightly different, the dialect interface allows vendors to create custom Hibernate mappings for a specific database. Internally, SQL queries are executed using the InterSystems IRIS TSQL dialect. See the [InterSystems IRIS TSQL Documentation](#) for details.

3.1 Requirements and Configuration

The following software must be installed on your system:

- InterSystems IRIS® 2026.1
- Hibernate 6.2 or 7.1
- A supported version of Java JDK 1.8 or higher (see “Supported Java Technologies” in the *InterSystems Supported Platforms* document for this release).

Hibernate will load the InterSystems dialect automatically when you specify the correct configuration information.

In the Hibernate configuration files (either `hibernate.properties` or `hibernate.cfg.xml`), specify the connection information for your database, and the name of the InterSystems dialect class.

The following five configuration properties are required:

- *dialect* — The fully qualified name of the InterSystems dialect class. The base dialect class is:

```
org.hibernate.dialect.InterSystemsIRISDialect
```

You can use a custom dialect class derived from this base class if you need to enable support for the Hibernate primary key generator classes.

- *driver_class* — The fully qualified name of the InterSystems JDBC driver class:

```
com.intersystems.jdbc.IRISDriver
```

This class is in the InterSystems JDBC driver .jar file.

- *username* — Username for the InterSystems IRIS namespace you want to access (default is `_SYSTEM`).
- *password* — Password for the InterSystems IRIS namespace (default is `SYS`).

- *url* — The URL for the InterSystems JDBC driver. The format for the URL is:

```
jdbc:IRIS://<host>:<port>/<namespace>
```

where *<host>* is the IP address of the machine hosting InterSystems IRIS, *<port>* is the SuperServer TCP port of your InterSystems IRIS instance, and *<namespace>* is the namespace that contains your InterSystems IRIS database data (see “[Defining a JDBC Connection URL](#)” in *Using Java with InterSystems Software* for more details).

A typical entry in `hibernate.properties` would contain the following lines (change `url`, `username`, and `password` as appropriate for your system)):

```
hibernate.dialect org.hibernate.dialect.InterSystemsIRISDialect
hibernate.connection.driver_class com.intersystems.jdbc.IRISDriver
hibernate.connection.url jdbc:IRIS://127.0.0.1:1972/USER/
hibernate.connection.username _SYSTEM
hibernate.connection.password SYS
```

The following example shows the same information as it would appear in `hibernate.cfg.xml`:

```
<hibernate-configuration>
  <session-factory>
    <property name="dialect">
      org.hibernate.dialect.InterSystemsIRISDialect
    </property>
    <property name="connection.driver_class">
      com.intersystems.jdbc.IRISDriver</property>
    <property name="connection.username">_SYSTEM</property>
    <property name="connection.password">SYS</property>
    <property name="connection.url">
      jdbc:IRIS://127.0.0.1:1972/USER
    </property>
  </session-factory>
</hibernate-configuration>
```

CAUTION: If the same property is set in both `hibernate.properties` and `hibernate.cfg.xml`, Hibernate will use the value from `hibernate.cfg.xml`.

4

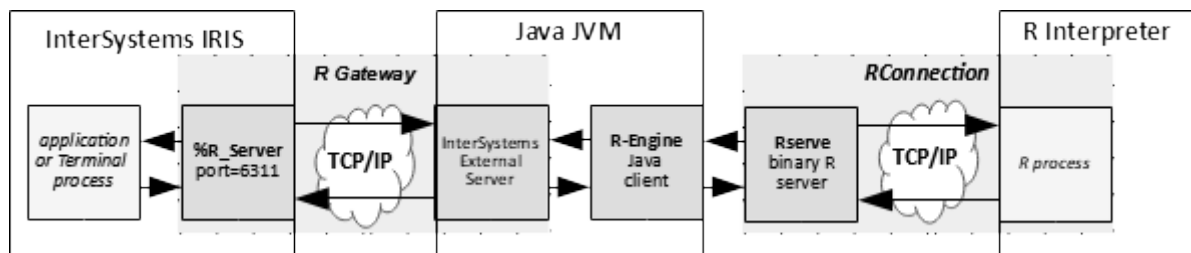
R Gateway Support

The InterSystems R Gateway is an extension of the Java [External Server](#) that enables a connection to an R interpreter through third party software. It incorporates two third party packages:

- *Rserve* is a binary package that provides a TCP/IP connection to an R interpreter.
- *R-Engine* is a Java client for Rserve.

These packages allow InterSystems IRIS to access the R interpreter via the InterSystems R Gateway. The R Gateway provides an interface to the R language that can be used in the same way as any other External Server language interface (see [Using InterSystems External Servers](#) for more information).

Figure 4-1: R Gateway System



Note: *Known Limitations*

Java differentiates empty string and null string, while the InterSystems IRIS database does not. The Java client maps both Java null and Java empty string to database empty string. InterSystems IRIS empty strings are mapped to Java null. To avoid problems, do not assign different meanings to empty and null strings.

4.1 Installing R and RServe

Requirements

- Java 8 or higher
- InterSystems IRIS 2020.4 or later. The following required resources are included with InterSystems IRIS:
 - the InterSystems %R_Server External Server is installed automatically (see “[Controlling Connections with the Management Portal](#)” in [Using InterSystems External Servers](#))

- the `org.rosuda.REngine` package is included in `intersystems-rgateway-<version>.jar` and used by the External Server.
- R interpreter (download a recent instance from <https://cloud.r-project.org/>)
- RServe (must be installed from the R Console, as described below)

Note: Although it has a similar name, the RGateway posted on GitHub (<https://github.com/intersystems-community/RGateway>) uses an older technology and is not supported by InterSystems.

Install R interpreter

Download and install the latest version of R: <https://cloud.r-project.org/>. Be sure to make R accessible to all users.

Install Rserve server

Rserve is a TCP/IP server which allows other programs to use facilities of R (see <http://www.rforge.net/Rserve/index.html>).

- Set environment variable `R_LIBS` to a directory accessible to all users. This is to make sure Rserve package is available to all users after installation. Details can be found at <https://cran.r-project.org/doc/manuals/R-admin.html#Managing-libraries>
- Start the R Console.
- In the R Console, run `install.packages("Rserve", , "http://rforge.net")`
- If prompted to install from sources, answer "Yes". Please refer to Rserve documentation (<https://www.rforge.net/Rserve/doc.html>) for more details.

4.2 Running the R Gateway

InterSystems IRIS connects to Rserve through an External Server gateway (see [Using InterSystems External Servers](#) for detailed information). To run the R Gateway, you must:

- [Launch Rserve](#) (either from the R Console or from ObjectScript, as described below).
- [Start the R Gateway](#) from the Management Portal.
- [Create the RConnection interface](#) between Rserve and the R interpreter.
- [Create an R Gateway](#) and use it to run R commands.

Launch Rserve

Rserve can be started manually on any host and port. It can also be launched programmatically from InterSystems IRIS on localhost.

To start Rserve manually, type the following commands in the R Console:

```
library(Rserve)
Rserve()
```

If unsuccessful, try `Rserve(args="--no-save --slave")`. Please refer to Rserve documentation (<https://www.rforge.net/Rserve/doc.html>) for more details.

To launch Rserve programmatically from IRIS, issue the following command:

```
Do
##class(%Net.Remote.Object).%ClassMethod(gateway, "com.intersystems.rgateway.Helper", "launchLocalServer",
port)
```

Start R Gateway

R Gateway can be started from Management Portal: System->Configuration->Connectivity->External Language Servers. A new R Gateway can also be created from this page.

See the %R Server entry in the *Configuration Parameter File Reference* for a description of the %R Server parameters found in the [Gateways] section of the CPF.

Create RConnection

RConnection is the interface between Rserve and the R interpreter. Each RConnection corresponds to a R session. Each R session has its own memory space. R sessions are not thread-safe. On UNIX machines, multiple RConnections can be created on a single port. On Windows machines, a single port can only support a single connection. Multiple Windows connections can be established using one port for each connection.

If Rserve is running, RConnection can be created directly:

```
set c = ##class(%Net.Remote.Object).%New(gateway, "org.rosuda.REngine.Rserve.RConnection")
```

If you are not sure whether Rserve is running, you can use the helper class to start it automatically. If Rserve is not running, it will try to start it locally and return an instance of RConnection:

```
set c =
##class(%Net.Remote.Object).%ClassMethod(gateway, "com.intersystems.rgateway.Helper", "createRConnection")
```

Note that the above calls can take additional parameters, such as host name, port number etc.

Create an R Gateway object and run R statements

You can create an R Gateway object from the default gateway or a named gateway:

- To get the default R Gateway (named "%R Server"): `set gateway = $SYSTEM.external.getRGateway()`
- To get a named R Gateway: `set gateway = $SYSTEM.external.getGateway("gateway name")`

When you create an R Gateway object, it provides an interface to the R language that can be used just like other External Server language interfaces (see [Using InterSystems External Servers](#) for more information).

`$SYSTEM.external` is an ObjectScript [External Server command interface](#) that simplifies using the R Gateway from an InterSystems terminal. For usage, type `do gateway.Help()` in the terminal.

